

C

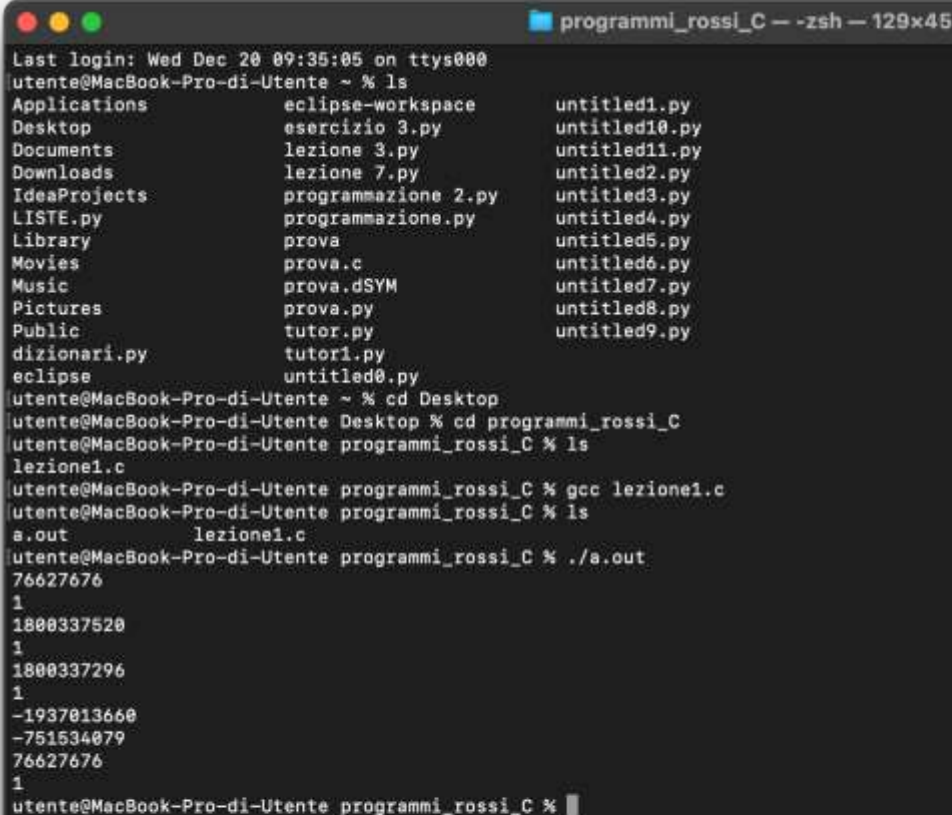
C è un linguaggio compilato, ciò significa che la macchina non comprende la roba scritta in C, quindi andremo ad usare il compilatore in modo tale che prenda il mio file in C e lo trasformi in un file eseguibile che anche la macchina possa capire.

Il compilatore di C si chiama GCC (Gnu C Compiler).

La sintassi per usare gcc nel terminale è la seguente:

```
gcc nome file.c
```

Ora vedremo passo passo come eseguire un file C:



```
programmi_rossi_C — -zsh — 129x45
Last login: Wed Dec 20 09:35:05 on ttys000
utente@MacBook-Pro-di-Utente ~ % ls
Applications      eclipse-workspace  untitled1.py
Desktop           esercizio 3.py    untitled10.py
Documents         lezione 3.py      untitled11.py
Downloads         lezione 7.py      untitled2.py
IdeaProjects      programmazione 2.py untitled3.py
LISTE.py          programmazione.py untitled4.py
Library           prova             untitled5.py
Movies            prova.c           untitled6.py
Music             prova.dSYM        untitled7.py
Pictures          prova.py          untitled8.py
Public            tutor.py          untitled9.py
dizionari.py      tutor1.py
eclipse           untitled0.py
utente@MacBook-Pro-di-Utente ~ % cd Desktop
utente@MacBook-Pro-di-Utente Desktop % cd programmi_rossi_C
utente@MacBook-Pro-di-Utente programmi_rossi_C % ls
lezione1.c
utente@MacBook-Pro-di-Utente programmi_rossi_C % gcc lezione1.c
utente@MacBook-Pro-di-Utente programmi_rossi_C % ls
a.out      lezione1.c
utente@MacBook-Pro-di-Utente programmi_rossi_C % ./a.out
76627676
1
1800337520
1
1800337296
1
-1937013660
-751534079
76627676
1
utente@MacBook-Pro-di-Utente programmi_rossi_C %
```

La "~" indica che sei nella "home"

Il comando **ls** ci permette di vedere quali file ci sono nella cartella directory in cui siamo

Il comando **cd** (change directory) ci permette di cambiare directory

Una volta che siamo all'interno della cartella dove c'è il nostro file C andremo ad eseguire **gcc** e andremo a creare il file eseguibile **a.out** che lo andremo ad eseguire con **./**

N.B: se noi sul terminale digitiamo **ls -l** avremo più informazioni complete sui file (da sinistra verso destra: nome file, orario modifica, la data, dimensione in byte, nome utente, nome gruppo a cui appartiene utente, una stringa che se letta a blocchi di tre ci dice che l'utente ha permesso di scrittura e lettura, se c'è un altro utente ci dice che gli altri utenti possono solo leggere)

cd .. torno indietro alla cartella precedente

- In C, ogni istruzione deve essere terminata da ; (**punto e virgola**).
- Chiameremo **identificatori**, cioè nomi dati alle variabili, funzioni e altre entità nel programma. Questi iniziano con una lettera o un carattere di sottolineatura (_) e possono essere seguiti da lettere, cifre o sottolineature
- Usiamo le parentesi graffe { } per delimitare i **blocchi di codice**.
- C fa **distinzione tra maiuscole e minuscole**. Ad esempio, variabile e Variabile sono considerate due variabili diverse.
- Ogni programma C deve contenere una funzione **main ()**. Questa funzione è il punto di ingresso del programma e viene eseguita quando il programma viene avviato.
- In C per scrivere un commento precediamo il testo con un doppio slash // per avere un **commento in una sola riga**. Per scrivere **commenti su più righe** usiamo la seguente sintassi
/* testo */

In C per creare una **variabile**, prima la devo definire e poi la posso inizializzare. Per definire una variabile bisogna definire il tipo

```
#include <stdio.h>

main ( ) {

int x;
x = 5;

}
```

È possibile modificare il valore di una variabile assegnandole un nuovo valore, ma una volta modificata il programma andrà a leggere solo la seconda assegnazione.

➤ PRINTF

printf () = stampa la stringa che inseriamo tra parentesi

Esempio:

```
#include <stdio.h>

int main ( ) {

    printf ( 'Hello Word! \n' );

}
```

[out]: Hello Word!

La funzione printf non è incorporata in C. Questa si trova in una libreria chiamata "stdio.h" o anche "standard di input output", ecco perché all'inizio del codice scriviamo

```
#include <stdio.h>
```

Nella pagina precedente, oltre a quello che vogliamo stampare dentro alla funzione printf abbiamo `\n` questo significa letteralmente "new line" e ci permette di andare a capo.

In questo caso abbiamo stampato una semplice frase, se invece vogliamo stampare il valore di una variabile all'interno della funzione printf dobbiamo usare `"%d"`, questo è un "**segnaposto**" che possiamo tradurre con il significato di "voglio stampare il valore di una variabile" poi separiamo questo da una virgola e andremo a scrivere il nome della variabile.

(N.B: se abbiamo `%d` sappiamo che il valore che si vuole prendere è un intero)

Ecco altri casi che possiamo trovare all'interno del printf:

<code>%(precisione)f</code>	per i float/double si deve specificare il formato
<code>%s</code>	per le stringhe
<code>%e</code>	per i decimali, in notazione esponenziale
<code>%c</code>	per i caratteri (char)
<code>%p</code>	per le variabili puntatore

scanf è un'altra funzione di input output come printf, ci permette di prendere l'input dall'utente.

Esempio: #include <stdio.h>

```
int main() {

    // Dichiarazione delle variabili
    int intero;
    float decimale;
    char carattere;
    double esponenziale;
    char stringa[100];
    int *puntatore;
    int variabile;

    // Assegnazione del puntatore
    puntatore = &variabile;

    // Prendere in input i dati
    printf ("Inerisci un numero intero: \n");
    printf ("Inserisci un numero decimale: \n");
    printf ("Inserisci un carattere: \n");
    printf("Inserisci un numero in notazione esponenziale: \n");
    printf("Inserisci una stringa: \n");
    printf("Inserisci un numero: \n");

    //Utilizzo scanf per leggere i diversi tipi di dati
    scanf ( "%d", &intero );
    scanf ( "%f", &decimale );
    scanf ("%c", &carattere );
    scanf ( "%s", stringa );
    scanf ("%lf", &esponenziale);
    scanf ( "%d", puntatore);

    // Stampare i dati
    printf (" Hai inserito: %d\n", intero);
    printf (" Hai inserito: %.2f\n", decimale);
    printf (" Hai inserito: %c\n", carattere);
    printf (" Hai inserito: %s\n", stringa);
    printf (" Hai inserito: %e\n", esponenziale);
    printf (" Hai inserito: %d tramite il puntatore\n", *puntatore);

    return 0;
}
```

[out]: Ti chiederà in input di inserire i vari tipi di dati e poi te li stamperà

➤ TIPI DI DATI E OPERATORI ARITMETICI

Tipi di dati numerici:

int: rappresenta numeri interi, positivi o negativi.

float: rappresenta i numeri a virgola mobile

short: un tipo di dati intero più breve rispetto a *int*.

long: un tipo di dati intero più lungo rispetto a *int*..

double: rappresenta numeri in virgola mobile a precisione doppia.

long double: rappresenta numeri in virgola mobile a precisione estesa.

Tipo di Dati Carattere:

char: rappresenta un singolo carattere ASCII o un piccolo numero intero.

Tipo di Dati Booleano:

_Bool (o *bool* con *<stdbool.h>*): rappresenta valori booleani true o false.

Gli **OPERATORI ARITMETICI** consentono di eseguire operazioni matematiche sui numeri. Ecco una lista degli operatori aritmetici principali in C:

+ (Addizione): Somma due numeri.

- (Sottrazione): Sottrae un numero da un altro.

* (Moltiplicazione): Moltiplica due numeri.

/ (Divisione): Divide un numero per un altro.

% (Resto della Divisione): Restituisce il resto di una divisione.

```
int somma = 10 + 5;    // somma conterrà 15
int differenza = 20 - 8; // differenza conterrà 12
int prodotto = 6 * 4;   // prodotto conterrà 24
int divisione = 15 / 3; // divisione conterrà 5
int resto = 17 % 4;     // resto conterrà 1
```

Gli **OPERATORI DI INCREMENTO E DECREMENTO** consentono di aumentare o diminuire il valore di una variabile.

Incremento ++

```
int contatore = 5;
contatore++; // Incremento postfisso, contatore conterrà ora 6
++contatore; // Incremento prefisso, contatore conterrà 7
```

Decremento --

```
int contatore = 10;
contatore--; // Decremento postfisso, contatore conterrà ora 9
--contatore; // Decremento prefisso, contatore conterrà 8
```

➤ CICLO WHILE

Il **ciclo while** è un'espressione che ripete un blocco di codice finchè la condizione specificata è vera. Ecco la sua sintassi:

```
while ( condizione ) {
```

Blocco di codice da eseguire finchè la condizione è vera

```
}
```

Quando la condizione è falsa andremo ad usare l'istruzione **else**.

Esempio: `#include <stdio.h>`

```
int main (){  
    int età = 15;  
    while (età <= 20 ) {  
        printf ("%d\n», età );  
        età ++  
    }
```

[out]: 15
16
17
18
19
20

➤ CICLO FOR

Il **ciclo for** è una struttura che consente di eseguire un blocco di codice un numero specifico di volte. Ecco la sua sintassi:

```
for ( inizializzazione; condizione; incremento/decremento ) {
```

Blocco di codice da eseguire durante ogni iterazione

```
}
```

Esempio: `#include <stdio.h>`

```
int main (){  
    for (int età; età <= 18; età ++ ) {  
        printf ("%d\n", età );  
    }  
}
```

[out]: i numeri da 0 a 18

➤ IF

If è un'espressione che consente di eseguire un blocco di codice solo se la condizione specificata è vera. Ecco la sua sintassi:

```
if ( condizione ) {
```

Blocco di codice da eseguire se la condizione è vera

```
}
```

Quando la condizione è falsa andremo ad usare l'istruzione **else**.

Esempio: `#include <stdio.h>`

```
int main (){
```

```
    int eta = 15;
```

```
    if (eta >= 18) {
```

```
        printf("Sei maggiorenne\n");
```

```
    } else {
```

```
        printf("Sei minorenne\n");
```

```
    }
```

```
}
```

[out]: Sei minorenne

➤ FUNZIONE

Una funzione è una porzione del programma che esegue un compito specifico. Ogni funzione ha un nome unico. In C bisogna dichiararla prima di utilizzarla, per farlo usiamo la seguente sintassi:

<i>Tipo</i>	<i>nome</i>	<i>(parametri);</i>
-------------	-------------	---------------------

```
Esempio:  #include <stdio.h>

int main (){

    int somma (int a, int b){
        return a + b
    }
}
```

Per chiamare una funzione basta utilizzare il suo nome seguito da parentesi tonde () che contengono gli argomenti della funzione (se presente).

```
Esempio:  #include <stdio.h>

// dichiarazione della funzione o prototipo della funzione
int somma(int a, int b);

int main ( ) {
    // Chiama la funzione somma
    int risultato = somma(5, 3);
    printf("La somma è: %d\n", risultato);
    return 0;
}

// Definizione della funzione
int somma(int a, int b) {
    return a + b;
}
```

[out]: la somma è: 8

➤ GLI ARRAY

Un **array** è una struttura dati che consente di memorizzare una raccolta di elementi dello stesso tipo. Ci consentono di gestire gruppi di dati in modo efficiente.

Ogni elemento in un array è identificato da un **indice**, che rappresenta la sua posizione nell'array. Gli indici degli array iniziano da zero.

Sono utili quando è necessario gestire un insieme di **dati simili**, come una serie di numeri, caratteri o altri oggetti dello stesso tipo.

A differenza di python, ha una **dimensione fissa**.

Per creare un array, basta dichiarare il suo tipo e specificare la dimensione desiderata.

Sintassi: *Tipo array nome array [dimensione array]*

```
Esempio:   #include <stdio.h>

             int main ( ) {
                 int numeri [ 10 ];
                 int i;
                 for ( i = 0; i < 10, i ++){
                     printf ( "%d/n", numeri [i] );
                 }
             }
```

[out]:

La dimensione dell'array non può essere il valore di una variabile.

Posso anche non specificare la dimensione dell'array, ad esempio

Sintassi: *Tipo array nome array [] = { elementi array }*

Esempio: `#include <stdio.h>`

```
int main ( ) {  
  
    int b[ ] = { 50, 1, 4, 9, 61};  
    int i;  
    printf ("b\n");  
  
    for ( i = 0; i < 5; i ++){  
        printf ("%d\n", b [ i ] );  
    }  
}
```

[out]:
b
50
1
4
9
61

Posso anche specificare sia la dimensione dell'array, sia definire solo alcuni valori dell'array. Da notare che il codice stampa gli elementi non definiti 0.

Esempio: `#include <stdio.h>`

```
int main ( ) {  
  
    int c [ 5 ] = { 3, 2, 10};  
    int i;  
    printf ("c\n");  
  
    for ( i = 0; i < 5; i ++){  
        printf ("%d\n", c [ i ] );  
    }  
}
```

[out]:
c
3
2
10
0
0

➤ I PUNTATORI

Un **puntatore** è una variabile che punta ad un'altra.

Solitamente ogni variabile ha un nome, un tipo, una locazione di memoria e un valore.

Invece, un puntatore è una variabile che, anziché contenere un qualsiasi numero, contiene un indirizzo in memoria. La sintassi di un puntatore è la seguente:

```
nome tipo * nome puntatore;
```

Esempio: `#include <stdio.h>`

```
int main ( ) {  
  
    int x = 3;    //dichiaro x  
    int *p;      //dichiaro il puntatore  
}
```

Abbiamo dichiarato un puntatore ad un intero. Ma questo puntatore non porta a nulla. Possiamo far puntare il puntatore alla variabile x, cioè in altre parole vogliamo che p assuma come valore l'indirizzo della variabile x. Per fare ciò basta fare così:

Esempio: `#include <stdio.h>`

```
int main ( ) {  
  
    int x = 3;  
    int *p;  
    p = &x;    //assegno al puntatore l'indirizzo di memoria della variabile x  
}
```

Adesso, il puntatore p "punta" alla variabile x. Ora andrò a stampare il contenuto della variabile puntata da p.

Esempio: `#include <stdio.h>`

```
int main ( ) {  
  
    int x = 3;  
    int *p;  
    p = &x;  
    printf ( "%d\n", *p );  
}
```

[out]: 3

➤ STRUCT

Se vogliamo scrivere un programma che ci rappresenta una linea formata da due coordinate su un piano cartesiano in C possiamo attraverso le struct. Una **struct** è un contenitore di un numero costante di tipi dato (o in altre parole collezione di variabili di diversi tipi di dati raggruppate in una singola unità). Questi dati posso essere diversi e una volta ultimata questa struct rappresenterà un nuovo tipo di dato detto **dato composto**. Viene definita nel seguente modo:

```
struct nome della struct {
```

Singoli elementi che compongono la struct //N.B: verranno chiamati campi

```
};
```

#RICORDA: Metti punto e virgola dopo la parentesi graffa chiusa (caso particolare)

Esempio: `#include <stdio.h>`

//creo una struct dove definisco come è strutturato un punto

```
struct Punto {  
float x;  
float y;  
};
```

```
int main (){
```

//creo i punti con le struct

```
struct Punto A;  
struct Punto B;
```

// assegno un campo (valore) alla struct A e B

```
A.x = 1;  
A.y = 1;
```

```
B.x = 5;
```

```
B.y = 4;
```

*/*Voglio calcolare le coordinate del punto medio tra i due punti quindi dichiaro la variabile Punto Medio con una struct */*

```
struct Punto Medio;
```

/ matematicamente sappiamo che questo è possibile facendo la somma delle coordinate/2 e la somma delle ascisse/2 */*

```
Medio.x = ( A.x + B.x )/2;
```

```
Medio.y = ( A.y + B.y )/2;
```

```
printf("(%f, %f)\n", Medio.x, Medio.y);  
}
```

[out]: (3.000000, 2.500000)

Posso Inizializzare una variabile di tipo struct, ciò significa che posso assegnare un valore iniziale a quella variabile al momento della sua dichiarazione. Per farlo inserirò i valori tra parentesi graffe (vedi esempio nella seguente pagina).

Esempio: `#include <stdio.h>`

```
struct punto {  
    float x;           //campi della struct  
    float y;  
};  
  
int main () {  
    struct punto p = { 1.0, 2.9 }; //inizializzazione della struct  
    p.x = 3.9;           //assegno alla struct p il campo x  
    p.y = p.x + 1;  
  
    printf ("%f %f\n", p.x, p.y);  
}
```

[out]: 3.900000 4.900000

La funzione **typedef** crea un nuovo nome per un tipo di dato già esistente. Per fare ciò usiamo la seguente sintassi.

struct *nome del tipo già esistente* *nome del nuovo tipo;*

Esempio: `#include <stdio.h>`

```
struct punto {  
    float x;           //campi della struct  
    float y;  
};  
  
typedef struct punto punto;  
  
int main () {  
    struct punto p = { 1.0, 2.9 };  
    punto q = p  
  
    p.x = 3.9;           //assegno alla struct p il campo x  
    p.y = p.x + 1;  
  
    printf ("%f %f\n", p.x, p.y);  
    printf ("%f %f\n", q.x, q.y);  
}
```

[out]: 3.900000 4.900000
1.000000 2.900000

➤ ALLOCAZIONE DINAMICA DELLA MEMORIA e FREE ()

Solitamente se vogliamo definire un array dobbiamo sempre dichiarare prima la sua dimensione, nonostante l'array sia ancora vuoto. Quindi se vogliamo, durante il nostro programma modificare la dimensione del nostro array dobbiamo allocare manualmente la memoria. Per fare ciò usiamo la funzione "malloc". Questa è una funzione della libreria <stdlib.h>

```
Esempio:  #include <stdlib.h>           //inseriamo la libreria per usare la funzione malloc
           #include <stdio.h>

           int main ( ) {
               int* x = malloc (sizeof (int));
               *x = 3;
               printf ( "%d \n", *x );
               free ( x );
           }
```

[out]: 3

Nel seguente codice abbiamo inizialmente allocato la memoria. Per farlo abbiamo inserito fuori dal main la libreria (stdlib.h) per poi chiamare la funzione malloc. Questa funzione vuole sapere quando deve allocare e per farlo usiamo sizeof (int), che in altre parole ci dirà "la dimensione in byte di un intero". Una volta allocata la memoria la malloc restituisce un puntatore alla memoria. A questo punto assegniamo il valore 3 alla nuova zona di memoria allocata, per farlo usiamo il puntatore referenziandolo (seguire la freccia) (*x) e stampiamo il valore.

Infine usiamo **free ()** per deallocare la sezione di memoria dinamica puntata da x (o in altre parole liberare la memoria allocata dinamicamente).

Spesso ci capiterà di allocare dinamicamente interi e array, ma possiamo allocare anche cose più complesse come ad esempio le strutture (struct).

```
Esempio:  #include<stdlib.h>
           #include<stdio.h>

           struct Punto {
               float x;
               float y;
           };

           int main ( ){
               struct Punto * p = malloc(sizeof(struct Punto));
           }
```

[out]:

➤ STRINGHE IN C

Le stringhe in C sono identificate in un tipo di dato chiamato "char" (c).

```
Esempio:  #include <stdio.h>

int main(){
    char c='A';
    printf("%c\n",c);
}
```

[out]: A

Un char però viene definito come un intero piccolo, infatti se al programma precedente provo a stampare il carattere A come un intero succederà la seguente cosa.

```
Esempio:  #include <stdio.h>

int main( ){
    char c = 'A';
    printf ("%c\n", c );
    printf ("%d\n", c );
}
```

[out]: A
65

65 è l'intero che se convertito in carattere ci da "A». Questa è la cosiddetta codifica ASCII.

Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	*	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	\$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

Dunque le stringhe non sono altro che array di caratteri.

Esempio1: `#include <stdio.h>`

```
int main (){  
    char c = ' A ' ;  
    char a[4] = "ciao";  
    printf ("%c\n", c);  
    printf ("%d\n", c);  
  
    printf ("%s\n", a);  
}
```

[out]: A
65
ciao

Esempio2: `#include <stdio.h>`

```
int main (){  
    char c = ' A ' ;  
    char a[4] = { ' c ' , ' i ' , ' a ' , ' o ' };  
    printf ("%c\n", c);  
    printf ("%d\n", c);  
  
    printf ("%s\n", a);  
}
```

[out]: A
65
ciao

N.B: RICORDA che quando si parla di char si usano apici singoli ' c ', mentre quando si parla di stringa si usano apici doppi "ciao".

➤ CASTING

Il casting dei dati in C è un processo che consente di convertire un tipo di dato in un altro. In C esistono 2 tipologie di casting:

- Casting Implicito

Avviene automaticamente quando si assegna un valore di un tipo di dati ad una variabile di un altro tipo di dati.

```
Esempio:  #include <stdio.h>

           int main( ){
               int x = 30;
               float y = x;
               printf ("%f\n", y );
           }
```

[out]: 30.000000

- Casting Esplicito

Viene eseguito manualmente dall'utente e bisogna usare un operatore di casting specifico per farlo. Usando la seguente sintassi:

<i>Tipo di dato 1</i>	<i>nome dato 1 =</i>	<i>valore;</i>
<i>Tipo di dato 2</i>	<i>nome dato 2 =</i>	<i>(tipo dato 2) nome dato 1;</i>

```
Esempio:  #include <stdio.h>

           int main( ){
               double x = 30;
               int y = ( int ) x;
               printf ("%d\n", c );
           }
```

[out]: 30